

文章笔记

文章笔记: Your Agent Needs a Harness, Not a Framework

基于 Inngest 官方博客整理

2026 年 4 月 2 日

你的 *Agent* 需要的是 *Harness* (线束), 而不是 *Framework* (框架)

作者/来源: Inngest

发布日期: 2025

预计阅读时长: 约 20 分钟

文章链接: <https://www.inngest.com/blog/your-agent-needs-a-harness-not-a-framework>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：什么是 Harness ?	2
1.1 Harness 与 Framework 的区别	2
1.2 本章小结	3
2 Utah：一个参考实现	3
2.1 整体架构	3
2.2 六个函数，而非一个单体	4
2.3 本章小结	4
3 Agent 循环的 Step 化	4
3.1 Think → Act → Observe 循环	4
3.2 Durable Execution 的直观理解	5
3.3 Tool 复用：不需要自己造轮子	6
3.4 本章小结	7
4 Sub-agent 委派与并发控制	7
4.1 Sub-agent：用 step.invoke() 派生子 Agent	7
4.2 Singleton 并发：一次只处理一个对话	8
4.3 本章小结	8
5 Context 管理：最难的问题	9
5.1 为什么 Context 管理如此困难?	9
5.2 两层 Context 剪枝	9
5.3 跨运行的 Session Compaction	9
5.4 本章小结	10
6 Multi-provider 支持与未来方向	10
6.1 Provider 无关的 LLM 抽象	10
6.2 尚未解决的问题	10
6.3 未来探索方向	11
6.4 本章小结	11
7 总结与延伸	11
7.1 核心思想回顾	11
7.2 对 Agent 开发者的启示	12
7.3 拓展阅读	12

1 引言：什么是 Harness？

在所有工程领域中，harness（线束/挽具）都指代同一种东西：**连接、保护和编排各组件的基础设施层**——它本身并不执行核心工作。汽车中的线束（wiring harness）在引擎、传感器和仪表盘之间路由信号；测试线束（test harness）为代码提供可重复执行和可观测的脚手架；安全挽具（safety harness）则在你坠落时保护你。

Inngest 的这篇博客提出了一个核心论点：**Agent 运行时同样需要一个 harness**。LLM 是引擎，Tool 是外设，Memory 是存储——但连接它们的是什么？当 LLM 在第五次迭代超时，谁来捕获这个失败？当两条消息冲突时，谁来防止竞态？当一个 webhook 事件到达时，谁来路由到正确的 handler？

核心论点：Harness vs. Framework

当前每个 Agent 框架都在从零构建自己的重试逻辑、状态持久化、任务队列和事件路由。然而，**持久化的事件驱动基础设施**（durable, event-driven infrastructure）已经解决了这些问题。将每次 LLM 调用或 Tool 调用建模为一个 **step**——一个可独立重试的工作单元——就是 harness 思维的核心。

文章的核心观点可以用一句话概括：**基础设施本身就是 harness**。框架关注的是 AI 逻辑的抽象，而 harness 关注的是让 AI 逻辑在生产环境中**可靠运行**的基础设施。

1.1 Harness 与 Framework 的区别

为了更清晰地理解二者的区别，下表从多个维度进行对比：

表 1: Harness 与 Framework 的对比

维度	Framework（框架）	Harness（线束）
关注点	AI 逻辑的抽象与组合	可靠运行的基础设施
重试机制	自行实现	内置 step 级重试
状态持久化	自行实现	自动持久化每个 step 结果
并发控制	自行实现	声明式 singleton 配置
可观测性	需要额外集成	step 级 trace 内置
事件路由	通常不涉及	核心能力
故障恢复	从头重跑或自定义逻辑	从上次成功的 step 恢复

类比理解：为什么叫“Harness”？

考虑汽车的线束（wiring harness）：它不是引擎，不是轮胎，但没有它，引擎的信号无法传递到仪表盘，传感器的数据无法到达 ECU。线束解决的是**连接和保护**的问题。同理，Agent harness 解决的不是“如何调用 LLM”的问题，而是“LLM 调用失败后怎么办”“多个请求同时到达怎么办”这类**基础设施层面**的问题。

1.2 本章小结

本章引入了 harness 的概念，指出当前 Agent 框架的一个共性问题：它们都在重复造轮子——重试、持久化、并发、事件路由——这些本质上是基础设施问题，不是 AI 问题。Inngest 的核心主张是用已有的 durable execution 基础设施来充当 Agent 的 harness。

2 Utah：一个参考实现

为了验证 harness 理念，Inngest 团队构建了 **Utah** (Universally Triggered Agent Harness) ——一个具备 Tool 调用、Memory、Sub-agent 委派和完整持久化能力的对话式 Agent。它使用 Telegram 或 Slack 作为前端，后端是最小化的 TypeScript 代码，没有使用任何 Agent 框架，仅依赖 Inngest 的 function、step 和 event 机制。

Utah 命名含义

Utah 的全称是 **Universally Triggered Agent Harness**。“Universally Triggered”强调 Agent 不关心自己是如何被激活的——可以是 Telegram webhook、Slack webhook、cron 定时任务、Sub-agent 调用或跨函数事件。触发方式与工作逻辑完全解耦。

2.1 整体架构

Utah 的架构有三个关键特征：

1. **事件驱动**：所有触发都通过 Inngest event 系统路由
2. **编排与 Agent 循环解耦**：编排层（Inngest Cloud）和执行层（本地 worker）分离
3. **本地 worker 无需公网端点**：通过 `connect()` API 建立 WebSocket 长连接

数据流如下：

表 2: Utah 数据流

步骤	描述
1	Telegram/Slack webhook 命中 Inngest Cloud
2	Webhook transform 将原始 HTTP payload 转换为类型化的 Inngest event
3	本地 worker 通过 WebSocket 接收 event，运行 Agent 函数
4	Agent 函数完成后，发出 reply event
5	独立的 reply 函数被触发，通过 Telegram/Slack API 发送回复

架构的关键洞见：触发解耦

Agent 循环不知道也不关心自己是被谁触发的。添加一个新的 Slack bot？只需配置一个新的 webhook transform 和一个对应的 reply 函数。Agent 循环本身**不需要任何改动**。这就是“universally triggered”的含义。

2.2 六个函数，而非一个单体

Utah 并非一个做所有事情的大函数，而是由**六个函数**通过事件通信组成：

表 3: Utah 的六个函数

函数	职责
handleMessage	主 Agent 循环
sendReply	向渠道发送回复（处理格式化和降级）
acknowledgeMessage	收到消息后立即发送 typing indicator
failureHandler	跨所有函数的全局错误处理
heartbeat	定期健康检查（cron 调度）
subAgent	通过 <code>step.invoke()</code> 隔离运行的 Sub-agent

这种分离的好处很明显：

- typing indicator 在收到消息时**立即**触发，不需要等待 Agent 循环完成
- reply 函数独立处理 Telegram/Slack 特有的格式转换和错误恢复（例如 LLM 输出了格式错误的 HTML 时降级为纯文本）
- failure handler 捕获所有函数的未处理错误，统一通知用户
- 每个函数有自己的重试策略、并发控制和触发条件

2.3 本章小结

Utah 作为 harness 理念的参考实现，展示了如何用事件驱动架构将 Agent 逻辑拆分为多个小型、专注的函数。触发解耦使得 Agent 可以支持任意通信渠道而无需修改核心逻辑，六函数设计使得每个关注点（循环、回复、错误处理、心跳、子任务）都可以独立配置和演进。

3 Agent 循环的 Step 化

3.1 Think → Act → Observe 循环

Utah 的核心是一个标准的 **think → act → observe** 循环。每次迭代调用 LLM，检查是否需要使用 Tool，执行 Tool，并将结果反馈回来。关键洞见是：**每次 LLM 调用和每次 Tool 执行都是一个 Inngest step。**

```
1 while (!done && iterations < config.loop.maxIterations) {
2   iterations++;
3
4   // Prune old tool results to keep context focused
5   pruneOldToolResults(messages);
6
7   // Budget warnings when running low on iterations
8   const messagesForLLM = addBudgetWarning(messages, iterations);
9
10  // Think: call the LLM
```

```

11  const llmResponse = await step.run("think", async () => {
12      return await callLLM(systemPrompt, messagesForLLM, tools);
13  });
14
15  const toolCalls = llmResponse.toolCalls;
16
17  if (toolCalls.length > 0) {
18      messages.push(llmResponse.message);
19
20      // Act: execute each tool as a separate step
21      for (const tc of toolCalls) {
22          const result = await step.run(`tool-${tc.name}`, async () => {
23              validateToolArguments(tool, tc);
24              return await executeTool(tc.id, tc.name, tc.arguments);
25          });
26
27          // Observe: feed results back into messages
28          messages.push(toolResultMessage(tc, result));
29      }
30  } else if (llmResponse.text) {
31      // No tools - the text response IS the reply
32      finalResponse = llmResponse.text;
33      done = true;
34  }
35  }

```

Listing 1: Utah Agent 循环核心逻辑（简化版）

Step 的三大关键特性

1. **自动索引**：当 `step.run("think")` 在循环中被调用十次时，Inngest 内部自动追踪为 `think:0, think:1, ...` 无需手动管理唯一 step ID。
2. **独立可重试**：如果 LLM API 在第 3 次迭代返回 500 错误，Inngest 只重试那一个 step。第 1、2 次迭代的结果已经持久化，不会重新执行。
3. **文本即结束**：当 LLM 返回文本而没有 tool call 时，这轮对话就结束了。无需显式的“done”信号。

3.2 Durable Execution 的直观理解

Durable execution（持久化执行）这个概念听起来抽象，但在 Agent 场景下非常直观：

没有 Durable Execution 时的典型故障

假设你的 Agent 正在执行一个 10 步的任务。在第 7 步时，LLM API 超时了。在传统架构下：

- 整个 Agent 循环崩溃
- 前 6 步的工作全部丢失
- 要么从头重跑（浪费时间和 token），要么自己实现 checkpoint 逻辑

而在 Inngest 的 step 模型下，前 6 步的结果已经持久化。重试只需要从第 7 步开始。这就是 durable execution 应用于 Agent 循环的价值。

这并不是什么新概念——durable execution 最初是为电商结账流程、支付处理等场景设计的。Inngest 的洞见是：Agent 循环本质上也是一个需要可靠执行的多步骤工作流。

3.3 Tool 复用：不需要自己造轮子

Utah 没有手写文件 I/O 和 shell 执行。它直接引入了 `pi-coding-agent`——来自 OpenClaw/Pi 生态系统的、经过实战检验的 Tool 实现：

表 4: Utah 使用的 Tool

Tool	功能
<code>read, write, edit</code>	文件操作（支持图片、二进制检测、智能截断）
<code>bash</code>	Shell 执行（可配置超时和输出截断）
<code>grep, find, ls</code>	搜索与导航（尊重 <code>.gitignore</code> ）
<code>remember</code>	将笔记持久化到每日日志
<code>web_fetch</code>	网页内容抓取
<code>delegate_task</code>	Sub-agent 委派（详见下节）

引入方式非常简洁：

```
1 import { createReadTool, createWriteTool, createBashTool }
2   from "@mariozechner/pi-coding-agent";
3
4 const tools = [
5   createReadTool(config.workspace.root),
6   createWriteTool(config.workspace.root),
7   createBashTool(config.workspace.root),
8   // ...custom tools
9 ];
```

Listing 2: Tool 引入示例

文章强调的观点是：AI Agent 的 Tool 与任何其他软件的依赖管理没有区别——使用现有的库，用 Inngest step 包装它们即可。

3.4 本章小结

将 Agent 循环中的每个操作建模为 Inngest step 是整篇文章最核心的技术设计。这种设计带来了自动重试、故障恢复和完整可观测性,且代码改动极小——只需将普通函数调用包装在 `step.run()` 中。Tool 层面,复用已有的开源实现而非重复造轮子,也是 harness 思维的体现。

4 Sub-agent 委派与并发控制

4.1 Sub-agent: 用 `step.invoke()` 派生子 Agent

有时 Agent 需要执行一个足够大的任务,大到会撑爆当前 context window——例如重构一个文件、调研一个课题、撰写一篇文档。对于像 OpenClaw 这样运行在单线程对话中的通用 Agent,长时间运行的会话可能因 context window 膨胀而出现问题。

解决方案: **派生 Sub-agent**。Utah 提供了 `delegate_task` tool。当主 Agent 调用它时,它使用 `step.invoke()` 启动一个完全独立的 Agent 函数运行实例。

```
1 // In the main agent loop, when delegate_task is called:
2 const subResult = await step.invoke("sub-agent", {
3   function: subAgent,
4   data: {
5     task: tc.arguments.task,
6     subSessionKey: `sub-${sessionKey}-${Date.now()}`,
7   },
8 });
```

Listing 3: 主 Agent 委派子任务

Sub-agent 的关键特征:

- 拥有自己独立的 context window (fork 父会话上下文到独立子会话)
- 使用相同的 Tool 集合 (但去除了 `delegate_task`——禁止递归派生)
- 拥有自己的重试策略和 step 级可观测性
- 完成后向父 Agent 返回一个摘要
- 父 Agent 只看到一个 Tool 结果: "这是我完成的内容"

```
1 export const subAgent = inngest.createFunction(
2   { id: "agent-sub-agent", retries: 1 },
3   { event: "agent.subagent.spawn" },
4   async ({ event, step }) => {
5     const { task, subSessionKey } = event.data;
6
7     const agentLoop = createAgentLoop(task, subSessionKey, {
8       tools: SUB_AGENT_TOOLS, // No delegate_task
9       isSubAgent: true,
10    });
11
12    return await agentLoop(step);
```

```
13 }  
14 );
```

Listing 4: Sub-agent 函数定义（简化版）

Sub-agent 的优雅之处

`step.invoke()` 本身就是 Inngest 的原生能力——它用于在一个函数中调用另一个函数、等待结果、然后继续执行。Sub-agent 并不需要任何特殊的“Agent-to-Agent 协议”。它只是函数调用函数。编排由基础设施自动处理。

4.2 Singleton 并发：一次只处理一个对话

每个通信渠道使用特定于渠道的 session key 来定义“对话”的边界。对于单线程渠道（如 Telegram），session key 是 chat ID；对于支持线程的平台（如 Slack），session key 包含 channel 和 thread 信息。

当多条消息在同一对话中发送时，你不希望第一个 Agent 循环继续运行，然后第二个循环再独立响应——你希望 Agent 拥有两条消息的上下文。Utah 选择的策略是：**取消当前运行，带着完整上下文重启**。

实现只需要一行配置：

```
1 singleton: { key: "event.data.sessionKey", mode: "cancel" },
```

Listing 5: Singleton 并发配置

这一行做了两件事：

1. **Singleton 并发**：以 `sessionKey` 为键，同一时刻每个对话只有一个 Agent 运行实例。没有竞态条件，没有交错响应。
2. **新消息时取消**：如果用户在 Agent 处理中发送新消息，当前运行被取消，新运行以最新消息启动。

Singleton 并发的权衡

取消 + 重启策略简洁，但有代价：任何正在进行中的工作都会丢失。新运行从持久化的 session 状态恢复，但这个过渡并不无缝。这是文章作者坦承的一个尚未完美解决的问题。在传统架构中，你需要自己构建每用户队列、管理锁和处理取消逻辑。Inngest 将其简化为一行配置，但底层问题仍然存在。

4.3 本章小结

Sub-agent 委派利用 Inngest 原生的 `step.invoke()` 实现，无需额外的 Agent 间通信协议。Singleton 并发控制通过一行声明式配置即可解决在传统架构中需要大量代码的竞态问题。这两个机制共同展示了 harness 思维的核心优势：**将复杂的编排问题下沉到基础设施层**。

5 Context 管理：最难的问题

5.1 为什么 Context 管理如此困难？

文章作者坦言：**最难的问题不是调用 LLM，而是管理送入 LLM 的内容。**

Utah 使用的 Tool 可能每次调用返回数千字符。经过几轮迭代后，对话上下文急剧膨胀，模型开始“迷失”——作者观察到 Agent 陷入无限循环地调用 Tool 却永远不产出最终回复。

Context 膨胀的恶性循环

Tool 返回大量内容 → context window 被填满 → 模型无法关注关键信息 → 模型反复调用 Tool 试图“理解”情况 → 更多内容涌入 → 恶性循环。这不是 LLM 的 bug，而是 context window 有限容量下的必然结果。**任何使用 Tool 的 Agent 都必须面对这个问题。**

5.2 两层 Context 剪枝

Utah 的解决方案是**两层 context pruning**：

```
1 const PRUNING = {  
2   keepLastAssistantTurns: 3,  
3   softTrim: {  
4     maxChars: 4000,  
5     headChars: 1500,  
6     tailChars: 1500  
7   },  
8   hardClear: {  
9     threshold: 50000,  
10    placeholder: "[Tool result cleared]"  
11  },  
12 };
```

Listing 6: Context 剪枝配置

具体策略如下：

表 5: Context 管理策略

策略	描述
Soft Trim	旧的 Tool 结果保留头部 1500 字符 + 尾部 1500 字符，中间截断
Hard Clear	当总 context 超过 50,000 字符时，旧的 Tool 结果完全替换为占位符
最近保留	最近 3 轮 assistant 的输出始终完整保留

5.3 跨运行的 Session Compaction

Pruning 处理的是**单次运行内的** context 管理。但 Agent 可能跨多次运行与用户持续对话。Session compaction 处理的是**跨运行**的上下文积累：当估算的 token 数超过阈值时，对话历史会被**总结压缩**后再送入下一次运行。

此外，Utah 还实现了两个辅助机制：

- **Budget Warning**: 当 Agent 接近最大迭代次数时，注入系统消息告诉它”请尽快收尾”
- **Overflow Recovery**: 如果 LLM 在运行中返回 context-too-large 错误，强制执行 compaction 并重试，不浪费迭代次数

Context 管理的四重防线

Utah 的 context 管理可以总结为四重防线：

1. **Pruning** (单次运行内)：裁剪旧的 Tool 结果
2. **Compaction** (跨运行)：总结压缩对话历史
3. **Budget Pressure** (预防性)：提前警告 Agent 收尾
4. **Overflow Recovery** (应急性)：context 超限时强制压缩并重试

这四重防线共同确保 Agent 不会因 context 膨胀而”迷失”。

5.4 本章小结

Context 管理是 Agent 工程中最被低估的挑战。它不是一个可以一次性解决的问题，而是需要多层防线协同工作的系统性方案。Utah 的实践表明，pruning、compaction、budget warning 和 overflow recovery 四管齐下，才能让 Agent 在长时间运行中保持稳定。

6 Multi-provider 支持与未来方向

6.1 Provider 无关的 LLM 抽象

Utah 不直接调用 Anthropic SDK，而是使用 `pi-ai`——一个 provider-agnostic 的 LLM 抽象层，支持 Anthropic、OpenAI 和 Google。切换 provider 只需改配置：

```
1 llm: {  
2   provider: "anthropic", // or "openai" or "google"  
3   model: "claude-sonnet-4-20250514",  
4 },
```

Listing 7: LLM Provider 配置

这种设计为未来的 Sub-agent 异构模型分配打下了基础——例如 coding Sub-agent 使用 Codex，research Sub-agent 使用 Opus。

6.2 尚未解决的问题

文章作者坦诚地列出了几个尚未完美解决的挑战：

表 6: Utah 当前的已知局限

问题	现状
Steering（转向）	新消息到达时 cancel+restart，进行中的工作丢失
Streaming	当前 step 是原子性的，不支持流式中间结果
实时更新	尚未利用 Inngest 的 realtime 功能推送循环中间进度

6.3 未来探索方向

Utah 目前是一个运行在本地机器或服务器上的单人 harness。Inngest 团队正在探索将其扩展为多人协作的方向：

- 1. **Swappable Sandboxes**: 可替换的沙箱环境，使 Utah 能在 serverless 环境运行
- 2. **External State and Memory**: 外部化状态和记忆存储
- 3. **Session Monitoring**: 基于 Inngest API 和 Insights 功能的 coding session 监控
- 4. **Human-in-the-Loop**: 使用 `step.waitForEvent()` 创建人工审批流程
- 5. **Self-building Agent**: 让 Utah 能够编写自身——创建新 Agent、新 workflow、新 webhook，并通过 API 监控自身

从 Single-player 到 Multi-player

Utah 当前的架构——事件驱动、触发解耦、函数组合——天然支持向多人协作方向演进。多个 Agent 可以通过事件系统通信，每个 Agent 的 sandbox 可以独立替换，状态可以外部化到共享存储。这不需要架构上的根本改变，只需要在现有基础设施上叠加新能力。

6.4 本章小结

Provider 无关的 LLM 抽象使得 Utah 能够灵活选择模型。文章坦诚地列出了当前的局限 (steering、streaming、realtime)，并描绘了从单人 harness 到多人协作平台的演进路线。这些方向进一步印证了 harness 思维的前瞻性——好的基础设施设计天然为未来扩展留出空间。

7 总结与延伸

7.1 核心思想回顾

这篇文章的核心论点可以用一个类比概括：

一句话总结

就像每辆汽车都需要线束来连接引擎、传感器和仪表盘一样，每个 AI Agent 都需要一个 harness 来连接 LLM、Tool 和 Memory。基础设施本身就是 harness——重试、持久化、并发控制、事件路由、可观测性——这些都是已经被解决的基础设施问题，不需要每个 Agent 框架从零重建。

文章通过 Utah 参考实现展示了这一理念的具体落地，核心技术点包括：

1. **Step 化 Agent 循环**: 每次 LLM 调用和 Tool 执行都是一个可独立重试、自动持久化的 step
2. **事件驱动架构**: 触发 (webhook) 与逻辑 (Agent loop) 完全解耦
3. **函数组合**: 六个小函数通过事件通信, 各司其职
4. **Sub-agent 委派**: 利用 `step.invoke()` 原生能力, 无需额外协议
5. **声明式并发控制**: 一行 singleton 配置解决竞态问题
6. **四重 Context 管理**: pruning + compaction + budget + overflow recovery

7.2 对 Agent 开发者的启示

这篇文章对正在构建 AI Agent 的开发者有几个重要启示:

1. **区分 AI 问题和基础设施问题**: prompt engineering、Tool 设计、context 策略是 AI 问题; 重试、持久化、并发、路由是基础设施问题。不要在 AI 代码中解决基础设施问题。
2. **优先选择成熟的基础设施**: durable execution、event-driven architecture、job queue 都是经过数十年验证的基础设施模式。Agent 开发不需要发明新轮子。
3. **可靠性不是事后添加的**: harness 不是”等 Agent 跑起来再加”的东西, 而应该是**第一天就考虑**的基础。
4. **Context 管理值得重点投入**: 这是作者”the hard way”学到的经验。多数 Agent 框架的教程不会告诉你这一点。

7.3 拓展阅读

- Utah 源码: <https://github.com/inngest/utah>
- Inngest 官方文档: <https://www.inngest.com/docs>
- OpenClaw / Pi Coding Agent: Utah 的 Tool 层来源
- pi-ai: provider-agnostic 的 LLM 抽象层
- Durable Execution 概念: Temporal、Inngest 等平台的核心范式